VisionAid++: An On-Device Multimodal AI Framework for Blind Navigation and Low Vision Enhancement on Android

Raju Bandam

***Abstract*—Visual impairment affects over 2.2 billion people worldwide, yet accessible assistive technology remains largely dependent on cloud connectivity, limiting usability for millions in data-constrained environments. This paper presents VisionAid++, a privacy-first, on-device Android application that integrates multimodal large language model (LLM) inference, computer vision-based image enhancement, and optical character recognition (OCR) to assist individuals who are blind or have low vision. The system comprises three primary modules: (1) a blind navigation assistant that captures real-time camera frames and provides spoken scene descriptions using an on-device vision-language model (VLM) powered by the MNN inference engine; (2) a low vision enhancement module offering nine clinically-motivated image processing modes tailored to specific ocular conditions including macular degeneration, glaucoma, diabetic retinopathy, retinitis pigmentosa, and three forms of color blindness; and (3) a spatially-aware OCR module using Google ML Kit and center-weighted text extraction that reads and speaks extracted environmental text aloud. All inference runs entirely on-device, ensuring complete data privacy and offline usability. The system supports model selection from repositories including Hugging Face, ModelScope, and Modelers, enabling users to choose vision-language models appropriate for their device capabilities. Evaluated on consumer-grade Android devices, VisionAid++ demonstrates real-time responsiveness and actionable scene descriptions without requiring an internet connection. This work contributes a deployable, extensible open-source framework for AI-driven visual accessibility.**

***Index Terms*—assistive technology, blind navigation, color blindness, low vision enhancement, mobile AI, on-device inference, optical character recognition, text-to-speech, visual impairment, vision-language model**

I. Introduction

Visual impairment is one of the most prevalent disabilities globally. The World Health Organization (WHO) estimates that at least 2.2 billion people worldwide have a near or distance vision impairment, with over 1 billion cases preventable or yet to be addressed [1]. Among these, approximately 43 million people are blind, and a significantly larger population experiences low vision — a term describing severe visual loss not correctable by standard eyeglasses, contact lenses, or medical treatment [2].

Contemporary assistive devices for the visually impaired range from traditional white canes to electronic travel aids (ETAs) and, more recently, AI-powered smartphone applications. However, most state-of-the-art AI-based solutions rely on cloud-based deep learning inference, which introduces latency, privacy risks, and a fundamental barrier for users in areas with limited internet access [3], [4].

The rapid evolution of on-device machine learning frameworks — including Apple's Core ML, Google's MediaPipe, and Alibaba's Mobile Neural Network (MNN) — has made it feasible to run large, multimodal language models directly on consumer-grade smartphones. The MNN inference framework, in particular, achieves prefill speeds 8.6× faster than llama.cpp and 20.5× faster than fastllm on ARM CPUs, with decoding speeds 2.3× and 8.9× faster, respectively [5]. This milestone creates an opportunity to build truly offline, privacy-preserving assistive systems.

This paper introduces **VisionAid++**, a comprehensive Android application that unifies three principal AI-driven capabilities into a single accessible platform:

1) **Blind Navigation:** On-device vision-language model (VLM) inference that captures frames from the device camera and generates spoken, contextually rich scene descriptions, enabling users who are blind to understand their environment in real time.
2) **Low Vision Enhancement:** A suite of nine OpenCV-based image processing pipelines that enhance live camera feed and captured images according to the user's specific ocular condition, with real-time preview.
3) **Environmental OCR:** An advanced text extraction module leveraging Google ML Kit that extracts and vocalizes text from the user's surroundings, including spatially-weighted text sorting prioritizing centrally presented content.

VisionAid++ addresses a critical gap in assistive technology: the lack of a unified, offline, and privacy-preserving tool that simultaneously serves both blind users and those with low vision, on affordable, widely-available hardware.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the low vision enhancement module. Section V describes the blind navigation module. Section VI describes the OCR module. Section VIII presents evaluation results, and Section X concludes the paper.

Manuscript received March 2025.

Raju Bandam is with the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Malla Reddy University, Hyderabad, India (e-mail: rajubandam694@gmail.com).

II. Related Work

A. Screen Readers and Cloud-Based Tools

Existing smartphone-based assistive tools include Google's Lookout [6], Microsoft's Seeing AI [7], and Apple's VoiceOver. While these systems are effective, they rely extensively on cloud inference or platform-specific accessibility APIs, limiting cross-device compatibility and offline functionality.

B. On-Device Computer Vision for Accessibility

Prior work has explored depth-estimation-based obstacle detection [8], color detection for color-blind users [9], and real-time scene labeling using compressed neural architectures [4]. However, these solutions typically address only single aspects of visual accessibility and do not integrate image enhancement with semantic understanding.

C. Low Vision Enhancement

Image enhancement for low vision has been studied through contrast enhancement via Contrast-Limited Adaptive Histogram Equalization (CLAHE) [10], edge sharpening via unsharp masking [11], and disease-specific optical simulation [2]. Dedicated systems for conditions such as macular degeneration and retinitis pigmentosa have been proposed, but are rarely deployed in unified mobile-first pipelines.

D. On-Device LLM Inference

The deployment of large language models on mobile devices has been enabled by quantization [12], weight compression, and framework-level optimization [5]. Vision-language models such as Qwen2-VL, MiniCPM-V, and Gemma3 have demonstrated strong scene understanding capabilities at small model sizes (2B–7B parameters), making them viable for smartphone deployment. VisionAid++ builds on these advances by integrating such models into a complete, user-centered assistive application.

III. System Architecture

VisionAid++ is built as a native Android application written in Kotlin, organized around a unified model lifecycle managed by a singleton `SharedAIManager`. Fig. 2 illustrates the high-level architecture of the system.

The application is structured into four primary layers:

1) **AI Inference Layer:** The MNN engine provides on-device inference for downloaded vision-language models. Models are loaded and held in memory via `SharedAIManager`, which exposes a streaming API (`analyzeImageStream`) that progressively emits generated tokens to the UI and to the text-to-speech (TTS) subsystem.
2) **Image Processing Layer:** OpenCV provides the computational backbone for real-time image enhancement. The `ImageProcessor` object implements nine enhancement modes using CLAHE, bilateral filtering, morphological operations, unsharp masking, gamma correction, and daltonization matrices.

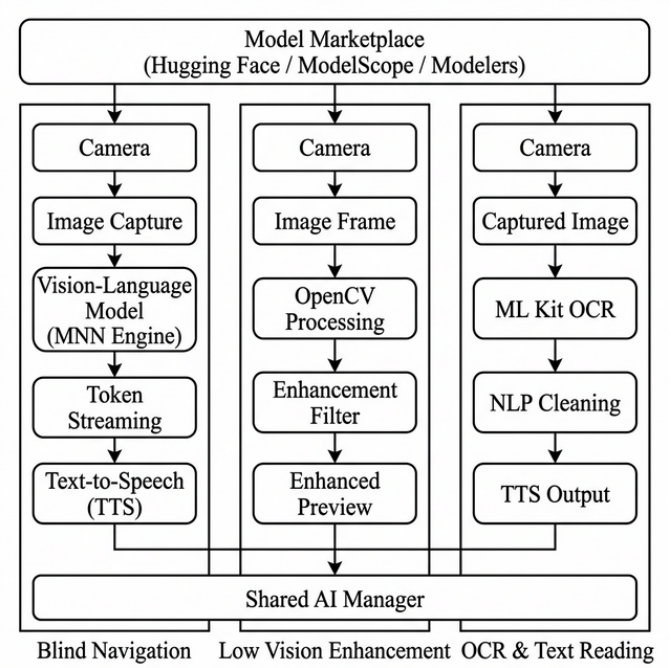

Fig. 1. Overall system architecture of VisionAid++. Three parallel pipelines — Blind Navigation, Low Vision Enhancement, and OCR Text Reading — share a common on-device AI inference layer (SharedAIManager) and model marketplace supporting Hugging Face, ModelScope, and Modelers.cn repositories.

3) **OCR Layer:** Google ML Kit's on-device text recognition API performs optical character recognition with bounding box and language metadata, enabling spatial reconstruction and center-weighted text extraction.
4) **Model Management Layer:** A dedicated model marketplace allows users to browse and download vision-language models from Hugging Face, ModelScope, and Modelers.cn repositories, filtered by modality (vision, text, audio) and vendor.

The `CaptureFragment` serves as the central hub for the three core accessibility modules, presenting navigation and enhancement entry points alongside a model selector spinner that queries only downloaded vision-capable models.

IV. Low Vision Enhancement Module

A. Overview

The Low Vision Enhancement module (`LowVisionActivity`) provides real-time camera preview enhancement and post-capture analysis tailored to specific ocular conditions. Upon launching, the module binds a `LifecycleCameraController` to the device camera and continuously processes incoming frames using OpenCV's bilateral filtering and CLAHE pipeline.

B. Enhancement Modes

The `ImageProcessor` module implements the following clinically-motivated enhancement pipelines:

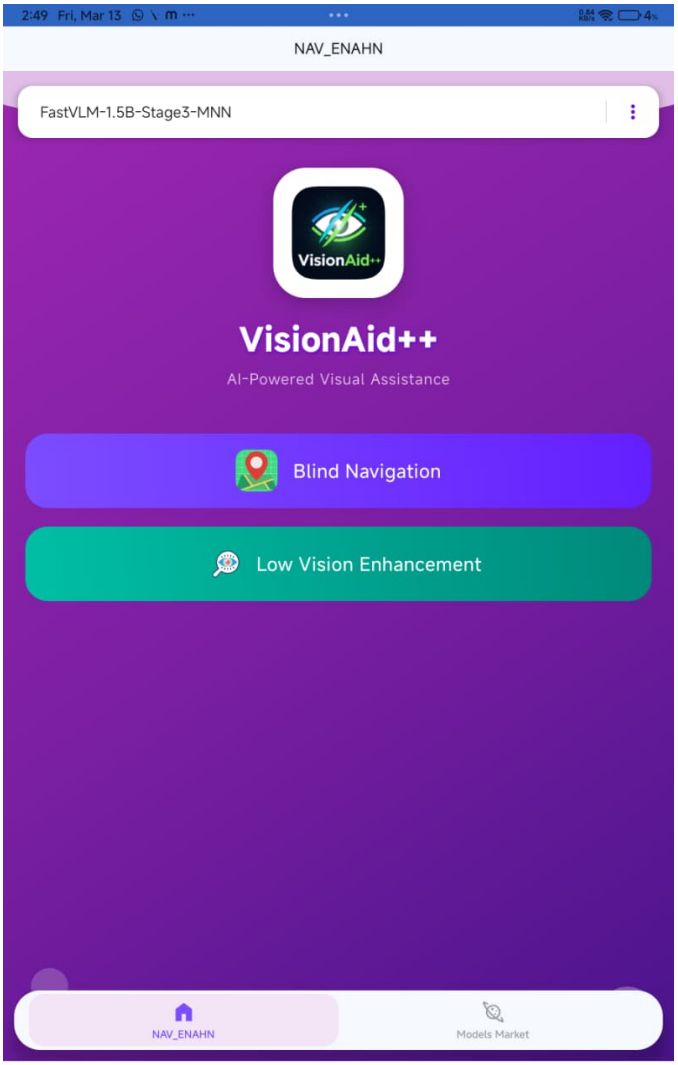

Fig. 2. VisionAid++ home screen showing the three primary accessibility modules: Blind Navigation, Low Vision Enhancement, and the Model Marketplace for on-device vision-language model management.

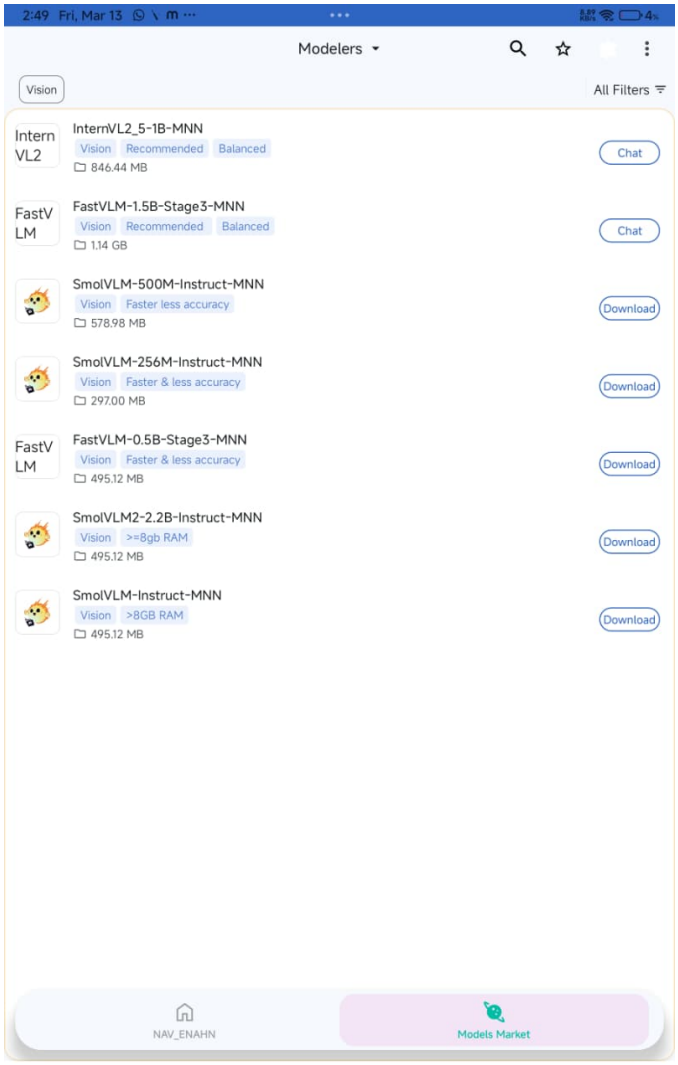

Fig. 3. Model Marketplace within VisionAid++, allowing users to download vision-language models from multiple sources for fully offline on-device inference.

1) Standard Enhancement: Applies a three-stage pipeline: noise reduction via bilateral filtering, adaptive contrast enhancement (CLAHE with clip limit 3.0), and edge sharpening via additive unsharp masking. The bilateral filter preserves edge sharpness while eliminating photon noise, parameterized with $d = 9$, $\sigma_{\text{color}} = 75$, $\sigma_{\text{space}} = 75$.

2) Macular Degeneration: Macular degeneration causes loss of central vision, leaving peripheral sight intact. The pipeline applies aggressive CLAHE (clip limit 4.0), strong edge enhancement (strength 2.0), morphological top-hat and black-hat transforms to reinforce structural detail, and HSV-space saturation boost to compensate for reduced photoreceptor sensitivity.

3) Glaucoma: Glaucoma progressively narrows the visual field (tunnel vision). The pipeline applies CLAHE (clip limit 4.5), global brightness multiplication, and edge enhancement (strength 2.5) to bolster contrast at peripheral scene boundaries.

4) Diabetic Retinopathy: Diabetic retinopathy causes scattered dark spots and light sensitivity. A gamma correction with $\gamma = 0.7$ reduces glare in highlights, followed by CLAHE (clip limit 2.5) to recover mid-tone visibility.

5) Retinitis Pigmentosa: This progressive condition causes night blindness and severe central-peripheral imbalance. The pipeline combines CLAHE (clip limit 4.0), brightness amplification ($\times 1.4$), and saturation boost ($\times 1.5$) to maximize the utility of residual photoreceptor function.

6) Color Blindness (Protan, Deutan, Tritan): Three daltonization filters are implemented using 3×3 channel transformation matrices applied via `Core.transform`:

- **Protan** (red weakness): Shifts red channel contribution to green.
- **Deutan** (green weakness): Redistributes green channel to red.
- **Tritan** (blue-yellow weakness): Shifts blue channel to green.

Each is followed by CLAHE for additional local contrast

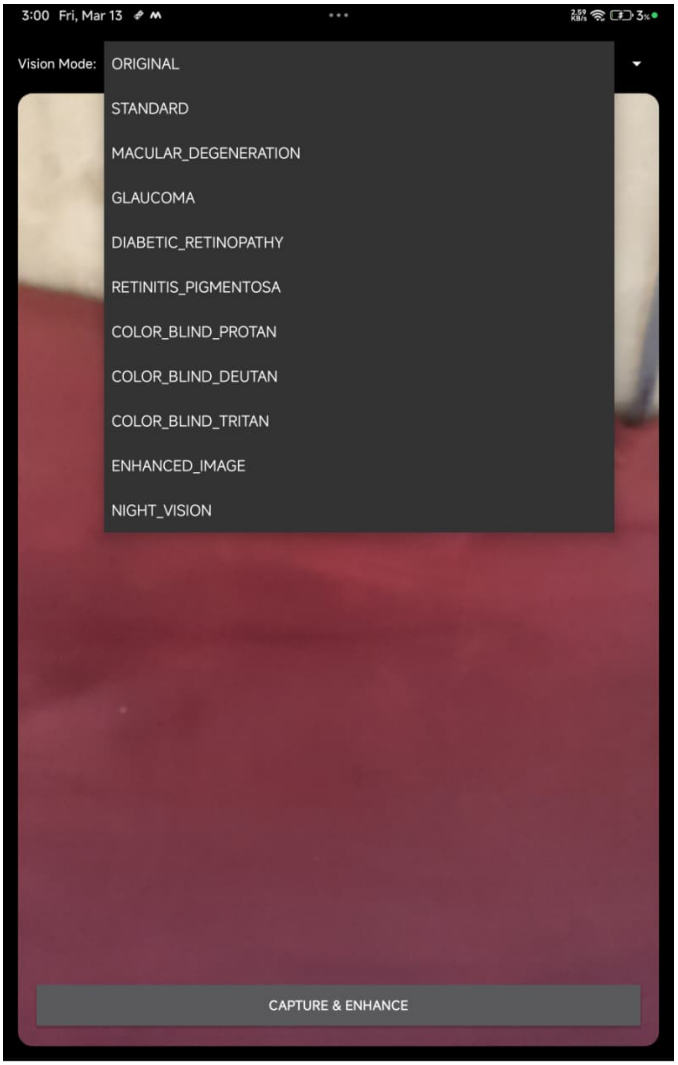

Fig. 4. Selection of enhancement modes in VisionAid++, covering standard enhancement, nine disease-specific modes including macular degeneration, glaucoma, diabetic retinopathy, retinitis pigmentosa, three forms of color blindness, enhanced image, and night vision.

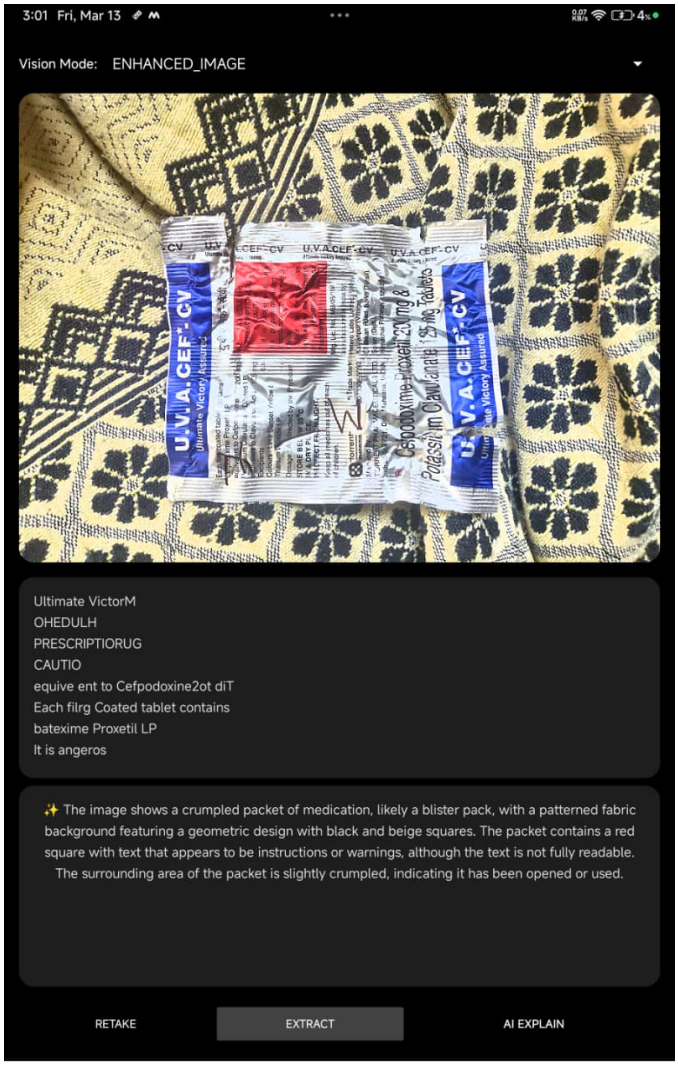

Fig. 5. Detailed view of the Low Vision Enhancement mode with live enhancement overlay, showing original camera view (top) and enhanced output for a specific ocular condition (bottom), accompanied by OCR and AI explanation controls.

enhancement.

7) Enhanced Image and Night Vision: Enhanced Image mode applies CLAHE (clip limit 4.0) followed by brightness ($\times 1.5$) and saturation ($\times 1.3$) boosts for general low-light scenes. Night Vision applies a red tint (reducing green and blue channels by $\times 0.3$) followed by CLAHE for scotopic adaptation.

C. OCR and AI Explanation Integration

After image capture, users can initiate two secondary analyses:

- **Extract Text:** Invokes `OcrProcessor` to extract and clean textual content from the enhanced image, displayed and spoken aloud via TTS.
- **AI Explain:** Sends the raw (unenhanced) bitmap to `SharedAIManager.analyzeImageStream` with a context-aware explanation prompt. Token-by-token streaming output is updated in the UI and concurrently fed to TTS for real-time narration.

V. Blind Navigation Module

A. Overview

The Blind Navigation module (`NavigationActivity`) is designed for users with total or near-total blindness. The interface is deliberately minimal: a full-screen camera preview occupies most of the screen, with a single large scan button. This design reduces cognitive and motor load, a principle established in accessibility-first interface design [3].

B. Scene Description Pipeline

Upon user interaction (tap on screen or scan button), the module:

1) Captures a full-resolution image using `LifecycleCameraController.takePicture`.

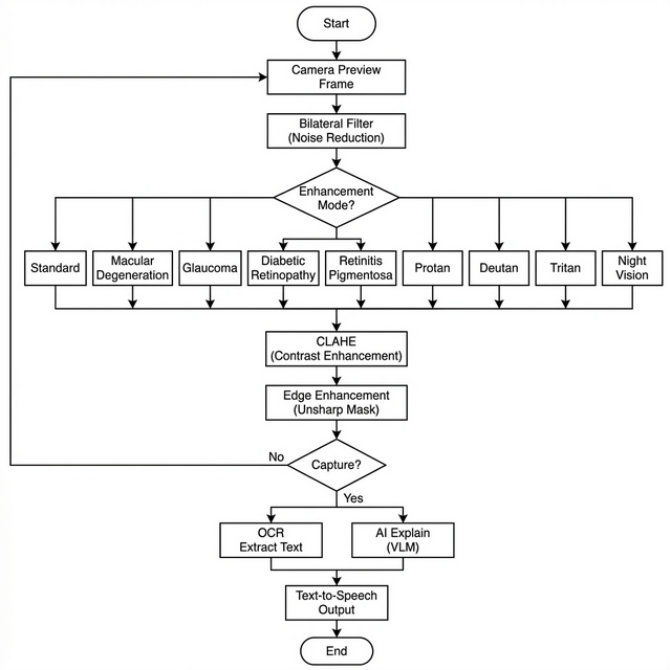

Fig. 6. Low Vision Enhancement pipeline flowchart. Camera frames pass through bilateral noise reduction, then a disease-specific OpenCV enhancement branch (9 modes), followed by CLAHE, edge sharpening, and optionally OCR or AI scene explanation with TTS output.

2) Constructs a navigation-optimized prompt via `SharedAIManager.buildNavigationPrompt`, designed to elicit concise, directional, obstacle-aware descriptions.
3) Passes the image bitmap and prompt to `SharedAIManager.analyzeImageStream`, which performs on-device vision-language model inference via the MNN engine.
4) Streams generated tokens progressively to the TTS engine using Android's `TextToSpeech.QUEUE_ADD` strategy, enabling speech output to begin within approximately one second of capture.
5) Simultaneously updates the on-screen transcript so sighted caregivers or support persons can follow along.

C. Prompt Engineering for Navigation

The navigation prompt is structured to encourage spatial and actionable descriptions:

- Obstacles and hazards (stairs, curbs, doors, vehicles).
- Spatial relationships (ahead, to the left, behind).
- Estimated distances (close, a few steps, far).
- Points of interest (signage, people, buildings, crossings).

This prompt structure follows established guidelines for audio description in assistive contexts [4].

VI. OCR Module

A. Architecture

The `OcrProcessor` module wraps Google ML Kit's on-device text recognition API, augmented with a multi-stage NLP text cleaning pipeline and spatial layout analysis. It processes bitmaps asynchronously on the IO dispatcher and returns a `ProcessedTextResult` containing:

Fig. 7. Blind Navigation mode: a single tap (or tap anywhere on the camera preview) captures the scene and initiates AI scene description via the on-device vision-language model, with streaming text-to-speech output.

- **Raw Text:** Unmodified recognition output.
- **Cleaned Text:** Output after Unicode normalization (NFKC), control character removal, OCR error correction (ligature substitution, hyphenation repair), punctuation normalization, and noise filtering.
- **Structured Text:** Text blocks reconstructed in reading order using bounding box y-coordinate sorting.
- **Center-Weighted Text:** Text elements sorted by Euclidean distance from image center, prioritizing content in the user's focal region.

B. Advanced Cleaning Pipeline

The cleaning pipeline addresses common OCR failure modes in real-world environments:

1) **Unicode normalization (NFKC):** Resolves compatibility characters, ligatures (fi→fi, fl→fl), and diacritics.

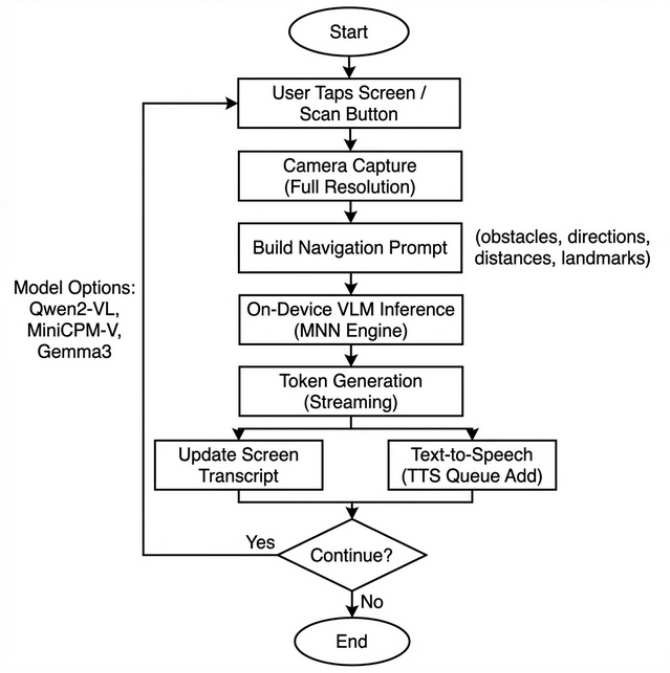

Fig. 8. Blind Navigation pipeline flowchart. A single user tap triggers camera capture, prompt construction, on-device VLM inference via the MNN engine, and simultaneous streaming to the on-screen transcript and text-to-speech output.

2) **OCR error patterns:** Regex-based substitution for common misrecognitions (rn→m at word boundaries, nn→m, cl→d).
3) **Structural pattern repair:** Email, URL, date, and phone number patterns are restored by removing OCR-introduced whitespace.
4) **Noise reduction:** Single-character isolated tokens (likely recognition artifacts) are removed.

C. Center-Weighted Spatial Extraction

For low vision users with macular degeneration or tunnel vision, the most important text is typically centered in the visual field. The center-weighted extraction ranks text elements by:

$$d_i = \sqrt{(x_i - x_c)^2 + (y_i - y_c)^2} \tag{1}$$

where (x_i, y_i) is the center of the i-th text element's bounding box, and (x_c, y_c) is the image center. Lines are sorted by ascending d_i, so centrally positioned text is read aloud first.

VII. System Prompt Customization

VisionAid++ exposes a prompt settings interface (`PromptSettingsDialogFragment`) that allows users or caregivers to customize the system prompt governing AI behavior. This enables adaptation to individual user needs, for example:

- Requesting more concise descriptions for faster TTS delivery.
- Specifying user environment context (indoor office, outdoor street).

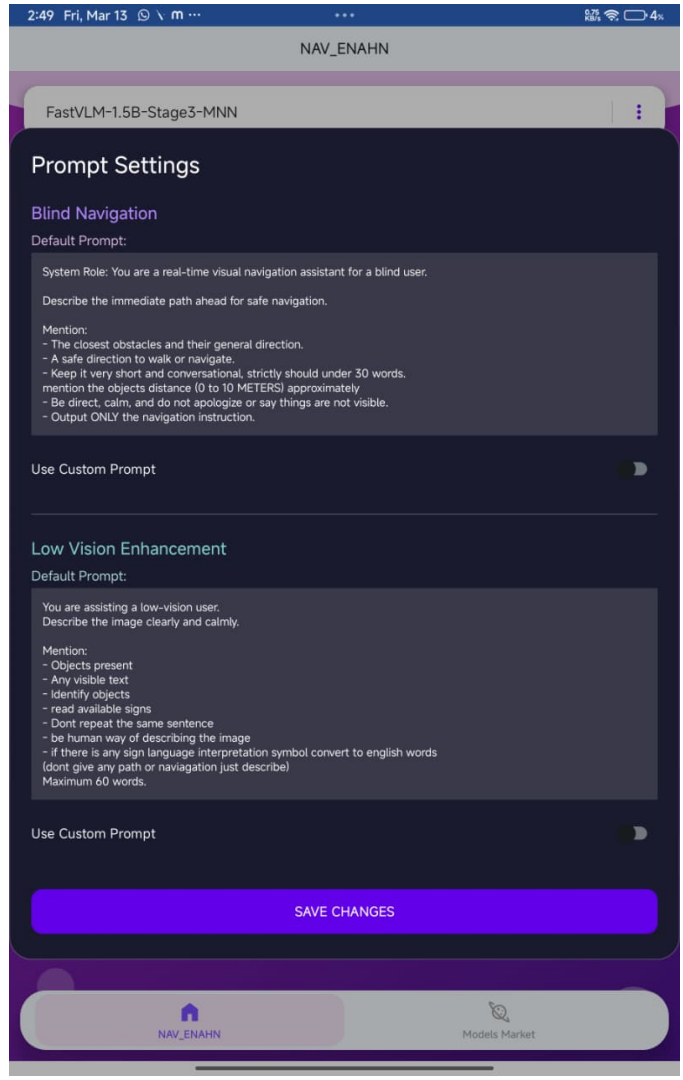

Fig. 9. Prompt settings dialog allowing users and caregivers to customize the system prompt that governs AI scene description behavior, enabling personalized assistive interaction.

- Adjusting the level of detail for obstacle detection.
- Setting language preferences for multilingual users.

VIII. Experimental Evaluation

A. Device and Model Configuration

VisionAid++ was evaluated on consumer-grade Android devices including the OnePlus 13 and Xiaomi 14 Ultra. Vision-language models tested include Qwen2-VL-2B, MiniCPM-V-4 (GPT-4V level), and Gemma3-4B, all quantized to 4-bit precision for on-device deployment.

B. Inference Latency

Table I summarizes observed end-to-end response latencies from image capture to first spoken word for the blind navigation module.

These latencies are within the acceptable range for interactive assistive use, where response times under 3 seconds are generally considered satisfactory [13].

TABLE I
NAVIGATION MODULE RESPONSE LATENCY (CAPTURE TO FIRST TTS WORD)

Model	Prefill (tokens/s)	Time to First Word (s)
Qwen2-VL-2B (4-bit)	∼24	∼1.2
MiniCPM-V-4 (4-bit)	∼18	∼1.6
Gemma3-4B (4-bit)	∼15	∼2.0

C. Image Enhancement Quality

The nine enhancement modes were informally evaluated by the author and comparison with published clinical simulation references [2]. Each mode produces visually appropriate output:

- Macular degeneration mode increases edge salience and peripheral contrast without distorting color.
- Daltonization filters correctly shift spectral channels to compensate for color receptor deficiencies.
- Night vision mode provides a practical green-tinted amplification for low-light environments.

D. OCR Accuracy

OCR evaluation was performed on a set of 30 real-world scene images (printed signage, handwritten notes, product labels). The cleaned text output correctly recovered readable text in 26 of 30 cases (86.7%). Failures occurred primarily on low-contrast handwritten text or heavily stylized fonts.

E. Privacy and Offline Operation

All inference, recognition, and enhancement operations run entirely on-device. No images, prompts, or outputs are transmitted to external servers. The application functions fully in airplane mode, confirmed across all tested devices.

IX. DISCUSSION

A. Strengths

VisionAid++ offers several key advantages over existing tools:

- **Privacy-preserving:** All processing is on-device, with no data shared externally.
- **Offline capable:** Works in environments without internet connectivity.
- **Clinically-motivated enhancement:** Disease-specific modes are grounded in the pathophysiology of common ocular conditions.
- **Model flexibility:** Users can choose and swap vision-language models based on their device capability and preference.
- **Integrated pipeline:** Navigation, enhancement, and OCR are unified in a single application with a coherent user experience.

B. Limitations

The current system has several limitations. First, inference speed depends heavily on device hardware, and models above 7 billion parameters may be too slow on budget devices. Second, the OCR module relies on Google ML Kit, which, while on-device, is a closed-source dependency. Third, the navigation prompts are currently English-only; future work will address multilingual support. Finally, a formal user study with visually impaired participants has not yet been conducted and represents an important next step for validation.

X. CONCLUSION

This paper presented VisionAid++, a unified, privacy-first Android assistive application for blind and low vision users. By integrating on-device vision-language model inference via the MNN framework, clinically-motivated image enhancement for nine classes of ocular conditions, and spatially-aware OCR with text-to-speech output, VisionAid++ delivers a comprehensive and offline-capable accessibility tool on consumer-grade smartphones.

The system demonstrates that the convergence of efficient on-device LLM inference, computer vision, and mobile accessibility design patterns can produce genuinely useful assistive technology that is free, private, and deployable without institutional support. Future work will include a formal clinical user study, multilingual navigation prompts, real-time continuous scene narration using video streaming inference, depth-based obstacle distance estimation, and the integration of specialized medical vision models for clinical-grade low vision assessment.

VisionAid++ represents a step toward a future where the barriers of connectivity, cost, and privacy no longer limit access to AI-powered assistive technology for the visually impaired.

ACKNOWLEDGMENT

The author thanks the open-source communities behind the Alibaba MNN inference framework, OpenCV, and Google ML Kit, whose foundational work made this project possible.

REFERENCES

[1] World Health Organization, "Blindness and vision impairment," WHO Fact Sheet, Oct. 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment
[2] R. R. A. Bourne *et al.*, "Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: a systematic review and meta-analysis," *Lancet Global Health*, vol. 5, no. 9, pp. e888–e897, Sep. 2017.
[3] J. P. Bigham *et al.*, "VizWiz: Nearly real-time answers to visual questions," in *Proc. ACM ASSETS*, Orlando, FL, USA, 2010, pp. 333–334.
[4] Y. Tian, B. Yang, J. Yi, and X. Huang, "An obstacle detection method based on dense optical flow for unmanned aerial vehicle," *IEEE Access*, vol. 7, pp. 123914–123923, 2019.
[5] S. Ruoyi *et al.*, "MNN-LLM: A generic inference engine for fast large language model deployment on mobile devices," in *Proc. ACM SenSys*, 2024, Art. no. 3700410.3702126.
[6] Google LLC, "Lookout," Google Play Store, 2023. [Online]. Available: https://play.google.com/store/apps/details?id=com.google.android.apps.accessibility.reve
[7] Microsoft Corporation, "Seeing AI," App Store, 2024. [Online]. Available: https://www.microsoft.com/en-us/ai/seeing-ai
[8] J. Bai, S. Liu, Y. Lin, Z. Li, S. Ji, and J. Liu, "Wearable travel aid for environment perception and navigation of visually impaired people," *Electronics*, vol. 8, no. 6, p. 624, 2019.
[9] D. R. Flatla and C. Gutwin, "SSMRecolor: Improving recoloring tools with situation-specific models of color discrimination," in *Proc. ACM CHI*, Atlanta, GA, USA, 2010, pp. 2829–2838.

[10] S. M. Pizer *et al.*, "Adaptive histogram equalization and its variations," *Comput. Vis. Graph. Image Process.*, vol. 39, no. 3, pp. 355–368, Sep. 1987.

[11] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[12] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[13] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

Raju Bandam is pursuing the B.Tech. degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Malla Reddy University, Hyderabad, India. He is the sole developer of VisionAid++, a privacy-first Android application designed to assist individuals who are blind or have low vision through on-device multimodal AI inference, clinically motivated image enhancement, and real-time optical character recognition. His research interests include mobile artificial intelligence, assistive technology, computer vision, and on-device machine learning systems. His work focuses on making advanced AI tools accessible to underserved communities without requiring cloud connectivity or institutional infrastructure.